

# SubTask-2

Tilak Asodariya

June 25, 2026

## Contents

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                                           | <b>2</b>  |
| <b>2</b>  | <b>Dataset Description</b>                                    | <b>2</b>  |
| <b>3</b>  | <b>Dataset Preprocessing</b>                                  | <b>3</b>  |
| <b>4</b>  | <b>Background</b>                                             | <b>3</b>  |
| 4.1       | Generative Adversarial Networks . . . . .                     | 3         |
| 4.2       | Deep Convolutional GAN (DCGAN) . . . . .                      | 4         |
| 4.3       | Spectral Normalization . . . . .                              | 4         |
| 4.4       | Wasserstein GAN with Gradient Penalty . . . . .               | 5         |
| <b>5</b>  | <b>Methodology</b>                                            | <b>5</b>  |
| 5.1       | Baseline Generative Adversarial Network . . . . .             | 5         |
| 5.1.1     | Generator . . . . .                                           | 5         |
| 5.1.2     | Discriminator . . . . .                                       | 6         |
| 5.2       | Deep Convolutional GAN (DCGAN) . . . . .                      | 6         |
| 5.2.1     | Generator . . . . .                                           | 6         |
| 5.2.2     | Discriminator . . . . .                                       | 7         |
| 5.3       | Training Strategy . . . . .                                   | 7         |
| 5.4       | Experiment 1: Spectral Normalization . . . . .                | 7         |
| 5.5       | Experiment 2: Improved Discriminator Architecture . . . . .   | 8         |
| 5.6       | Experiment 3: Wasserstein GAN with Gradient Penalty . . . . . | 8         |
| 5.7       | Experimental Hyperparameters . . . . .                        | 9         |
| <b>6</b>  | <b>Experimental Results</b>                                   | <b>9</b>  |
| 6.1       | Baseline GAN . . . . .                                        | 9         |
| 6.2       | Deep Convolutional GAN . . . . .                              | 10        |
| 6.3       | Experiment 1: Spectral Normalization . . . . .                | 11        |
| 6.4       | Experiment 2: Improved Discriminator . . . . .                | 12        |
| 6.5       | Experiment 3: WGAN-GP . . . . .                               | 13        |
| <b>7</b>  | <b>Loss Analysis</b>                                          | <b>14</b> |
| <b>8</b>  | <b>Comparative Analysis</b>                                   | <b>14</b> |
| <b>9</b>  | <b>Conclusion</b>                                             | <b>16</b> |
| <b>10</b> | <b>Future Work</b>                                            | <b>16</b> |

# 1 Introduction

Generative models attempt to learn the probability distribution of a dataset so that new samples resembling the original data can be synthesized. Unlike supervised learning, where labels guide the learning process, generative modeling focuses on understanding the intrinsic structure of data.

Among numerous generative approaches, Generative Adversarial Networks (GANs), introduced by Goodfellow et al., have demonstrated remarkable success in image synthesis, super-resolution, image translation, and data augmentation. GANs formulate image generation as a competitive game between two neural networks:

- Generator ( $G$ ): Produces synthetic images from random latent vectors.
- Discriminator ( $D$ ): Distinguishes real images from generated images.

During training, the Generator attempts to fool the Discriminator by producing increasingly realistic images, while the Discriminator continually improves its ability to identify fake images. This adversarial optimization enables the Generator to approximate the true data distribution without explicitly defining it.

Although the original GAN framework is conceptually elegant, practical training is often unstable due to issues such as mode collapse, vanishing gradients, and imbalance between the Generator and Discriminator. Over the years, numerous improvements have been proposed to address these challenges.

The primary objective of this assignment is to investigate how different architectural modifications influence GAN performance. Starting from a simple baseline implementation, progressively more sophisticated architectures were explored to improve both image quality and training stability.

## 2 Dataset Description

All experiments were conducted using the CIFAR-10 dataset.

CIFAR-10 is a benchmark image classification dataset containing natural images from ten object categories. Each image has a spatial resolution of  $32 \times 32$  pixels with three RGB color channels.

The dataset consists of:

| Property         | Value          |
|------------------|----------------|
| Training Images  | 50,000         |
| Testing Images   | 10,000         |
| Image Resolution | $32 \times 32$ |
| Channels         | RGB (3)        |
| Classes          | 10             |
| Total Images     | 60,000         |

Table 1: CIFAR-10 Dataset Statistics

The ten object categories are:

- Airplane
- Automobile
- Bird

- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

Unlike image classification tasks, GAN training ignores class labels and instead learns the complete distribution of natural images.

### 3 Dataset Preprocessing

The CIFAR-10 dataset is distributed as serialized Python batch files. Each batch contains image tensors and corresponding labels. Before training, all batches were unpacked, combined, and converted into tensors suitable for PyTorch.

Images were reshaped into the format

$$3 \times 32 \times 32$$

where the three channels correspond to Red, Green, and Blue respectively.

The following preprocessing operations were applied:

- Conversion from unsigned integers to floating point tensors.
- Pixel normalization from  $[0, 255]$  to  $[0, 1]$ .
- Rescaling from  $[0, 1]$  to  $[-1, 1]$  using

$$x' = 2x - 1$$

to match the output range of the Generator's Tanh activation.

- Creation of custom PyTorch datasets and dataloaders.
- Random shuffling during every training epoch.

Normalizing images to  $[-1, 1]$  significantly improves optimization stability because the Generator also produces outputs within the same numerical range.

## 4 Background

### 4.1 Generative Adversarial Networks

A standard GAN consists of two neural networks trained simultaneously.

The Generator receives a latent vector sampled from a Gaussian distribution,

$$z \sim \mathcal{N}(0, 1)$$

and transforms it into a synthetic image.

The Discriminator receives either a real or generated image and predicts the probability that the image belongs to the real dataset.

The optimization objective is formulated as the minimax game

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_z [\log(1 - D(G(z)))].$$

During training,

- the Discriminator learns to maximize classification accuracy,
- the Generator learns to fool the Discriminator.

This continuous competition drives both networks toward better performance.

## 4.2 Deep Convolutional GAN (DCGAN)

The original GAN employed fully connected layers, which ignored the spatial relationships between neighboring pixels.

DCGAN replaces these dense layers with convolutional operations, enabling the network to learn hierarchical image features.

Major architectural improvements include:

- Convolutional discriminator
- Transposed convolution generator
- Batch Normalization
- LeakyReLU activations
- Tanh output layer

Rather than generating every pixel independently, DCGAN progressively constructs images through multiple stages of learned upsampling, resulting in significantly improved visual quality.

## 4.3 Spectral Normalization

One of the major causes of GAN instability is an excessively powerful discriminator. If the discriminator becomes too accurate early in training, the Generator receives very small gradients and learning slows considerably.

Spectral Normalization constrains the spectral norm of each weight matrix, effectively limiting the Lipschitz constant of the discriminator.

Instead of allowing unrestricted weight growth,

$$W \rightarrow \frac{W}{\sigma(W)}$$

where  $\sigma(W)$  denotes the largest singular value.

This normalization prevents unstable gradient magnitudes while maintaining discriminator expressiveness, resulting in smoother adversarial training.

#### 4.4 Wasserstein GAN with Gradient Penalty

The original GAN objective relies on the Jensen-Shannon divergence, which often produces unstable optimization and vanishing gradients.

WGAN replaces this objective with the Earth Mover (Wasserstein) distance, providing a smoother optimization landscape.

Instead of a binary discriminator, WGAN employs a critic that assigns unrestricted real-valued scores to images.

To satisfy the Lipschitz constraint without weight clipping, WGAN-GP introduces the Gradient Penalty term

$$\lambda(\|\nabla D(\hat{x})\| - 1)^2$$

which regularizes the gradient norm along interpolated samples.

This modification substantially improves convergence stability and reduces mode collapse.

## 5 Methodology

This section describes the implementation details of all models explored in this assignment. The work began with a simple fully-connected GAN to establish a baseline. The baseline was subsequently improved using convolutional architectures, followed by three independent architectural modifications to improve image quality and training stability.

### 5.1 Baseline Generative Adversarial Network

The baseline model follows the original GAN framework proposed by Goodfellow et al. Both the Generator and Discriminator were implemented using fully-connected neural networks.

#### 5.1.1 Generator

The Generator receives a 100-dimensional latent vector sampled from a standard Gaussian distribution,

$$z \sim \mathcal{N}(0, 1)$$

and transforms it into a flattened image using a sequence of linear layers.

The architecture is summarized below.

| Layer         | Output Dimension        |
|---------------|-------------------------|
| Input Noise   | 100                     |
| Linear + ReLU | 256                     |
| Linear + ReLU | 512                     |
| Linear + ReLU | 1024                    |
| Linear + Tanh | $3 \times 32 \times 32$ |

Table 2: Baseline Generator Architecture

The Tanh activation constrains pixel values to the interval

$$[-1, 1]$$

which matches the preprocessing applied to the training images.

### 5.1.2 Discriminator

The Discriminator consists of a multi-layer perceptron that receives a flattened image and predicts whether it belongs to the real dataset.

| Layer              | Output Dimension |
|--------------------|------------------|
| Input Image        | 3072             |
| Linear + LeakyReLU | 1024             |
| Linear + LeakyReLU | 512              |
| Linear + LeakyReLU | 256              |
| Linear + Sigmoid   | 1                |

Table 3: Baseline Discriminator Architecture

Binary Cross Entropy (BCE) loss was employed for adversarial optimization.

Although this architecture successfully learns the adversarial objective, fully-connected layers ignore spatial relationships between neighboring pixels. Consequently, generated samples exhibit blurry textures and poor object structure.

## 5.2 Deep Convolutional GAN (DCGAN)

To overcome the limitations of dense networks, the baseline architecture was replaced with the Deep Convolutional GAN (DCGAN).

Instead of generating every pixel independently, convolutional operations allow the network to learn hierarchical spatial features, resulting in more realistic images.

### 5.2.1 Generator

The Generator begins from a latent vector of dimension 100.

Instead of immediately producing a complete image, the latent vector is first reshaped into a feature map of size

$$100 \times 1 \times 1.$$

Successive transposed convolution layers progressively increase the spatial resolution while simultaneously reducing the number of feature channels.

The architecture is shown in Table 4.

| Layer         | Feature Maps | Output Size    |
|---------------|--------------|----------------|
| Input Noise   | 100          | $1 \times 1$   |
| ConvTranspose | 512          | $4 \times 4$   |
| ConvTranspose | 256          | $8 \times 8$   |
| ConvTranspose | 128          | $16 \times 16$ |
| ConvTranspose | 3            | $32 \times 32$ |

Table 4: DCGAN Generator

Batch Normalization follows every transposed convolution except the final layer.

ReLU activation is used throughout the network, while the output layer employs Tanh activation.

Progressive upsampling enables the Generator to learn both global object structure and local image details.

### 5.2.2 Discriminator

The Discriminator performs the reverse operation.

Instead of flattening images immediately, convolutional layers gradually compress spatial information while increasing feature depth.

The architecture consists of:

- Convolution
- Batch Normalization
- LeakyReLU
- Sigmoid Output

Unlike the baseline model, convolutional feature extraction preserves local texture information and object boundaries.

### 5.3 Training Strategy

Training follows the standard adversarial optimization procedure.

For every mini-batch:

1. Sample real images from the dataset.
2. Generate fake images using random latent vectors.
3. Update the Discriminator using both real and fake samples.
4. Freeze the Discriminator.
5. Update the Generator using gradients propagated through the Discriminator.

The Discriminator learns to correctly classify real and generated images, whereas the Generator continuously improves by attempting to fool the Discriminator.

This alternating optimization continues until equilibrium is reached.

### 5.4 Experiment 1: Spectral Normalization

The first architectural improvement focuses on stabilizing the Discriminator.

A common issue during GAN training is that the Discriminator rapidly becomes significantly stronger than the Generator. When this occurs, gradients reaching the Generator become extremely small, slowing or even preventing learning.

To address this issue, Spectral Normalization was incorporated into every convolutional layer of the Discriminator.

Instead of allowing unrestricted weight magnitudes,

$$W \rightarrow \frac{W}{\sigma(W)}$$

where  $\sigma(W)$  represents the largest singular value of the weight matrix.

This normalization constrains the Lipschitz constant of each layer, preventing excessively large gradients while maintaining discriminator capacity.

Only the Discriminator architecture was modified.

To ensure a fair comparison, the pretrained DCGAN Generator was loaded from the baseline checkpoint and fine-tuned alongside the new Spectral Normalized Discriminator for twenty additional epochs.

This experimental design isolates the effect of Spectral Normalization without changing the Generator architecture.

## 5.5 Experiment 2: Improved Discriminator Architecture

The second experiment investigates whether increasing the representational capacity of the Discriminator improves image generation.

Instead of modifying the Generator, additional convolutional feature extraction was introduced within the Discriminator.

The modified architecture includes:

- Additional convolution layer
- Increased feature depth
- Batch Normalization
- Dropout regularization

The additional convolutional block enables deeper hierarchical feature extraction before making the final real-versus-fake decision.

Dropout regularization was incorporated to reduce overfitting, preventing the Discriminator from memorizing the training images and encouraging the Generator to learn more robust image representations.

As in the previous experiment, the pretrained DCGAN Generator was reused while only the Discriminator architecture was modified.

The Generator was subsequently fine-tuned for twenty epochs against the improved Discriminator.

## 5.6 Experiment 3: Wasserstein GAN with Gradient Penalty

The final experiment replaces the original GAN objective entirely.

Instead of predicting probabilities, the Discriminator is replaced with a Critic that assigns unrestricted real-valued scores to images.

Consequently, the Sigmoid activation and Binary Cross Entropy loss are removed.

The Critic objective becomes

$$L_C = -\mathbb{E}[C(x)] + \mathbb{E}[C(G(z))] + \lambda GP$$

where  $GP$  denotes the gradient penalty term.

The Generator minimizes

$$L_G = -\mathbb{E}[C(G(z))]$$

which encourages generated images to receive increasingly higher critic scores.

Unlike the previous experiments, the Critic is updated five times for every Generator update.

This strategy allows the Critic to remain close to its optimum, producing more informative gradients for Generator optimization.

Gradient Penalty further enforces the Lipschitz constraint by regularizing the gradient norm over interpolated samples.

Compared to weight clipping used in the original WGAN, Gradient Penalty significantly improves optimization stability while reducing mode collapse.



## 5.7 Experimental Hyperparameters

Table 5 summarizes the primary hyperparameters used throughout the experiments.

| Parameter          | Value              |
|--------------------|--------------------|
| Dataset            | CIFAR-10           |
| Image Resolution   | $32 \times 32$     |
| Latent Dimension   | 100                |
| Batch Size         | 128                |
| Optimizer          | Adam               |
| Learning Rate      | $1 \times 10^{-4}$ |
| Adam Betas         | (0.5,0.999)        |
| Baseline Epochs    | 50                 |
| Fine-tuning Epochs | 20                 |

Table 5: Training Hyperparameters

For WGAN-GP, the following additional hyperparameters were employed.

| Parameter        | Value     |
|------------------|-----------|
| Critic Updates   | 5         |
| Gradient Penalty | 10        |
| Adam Betas       | (0.0,0.9) |

Table 6: WGAN-GP Hyperparameters

## 6 Experimental Results

This section presents the qualitative and quantitative observations obtained from each implemented model. Since the primary objective of generative modeling is to synthesize visually realistic images, qualitative evaluation through generated samples forms the major component of the analysis. Training behaviour was additionally monitored using Generator and Discriminator (or Critic) losses.

### 6.1 Baseline GAN

The baseline fully-connected GAN successfully learned the adversarial objective and generated images resembling the CIFAR-10 data distribution. However, the generated samples exhibited several shortcomings.

- Poor spatial coherence.
- Blurry textures.
- Weak object boundaries.
- Difficulty learning global image structure.

The primary limitation arises from the use of fully-connected layers, which treat every pixel independently and fail to exploit local spatial correlations.

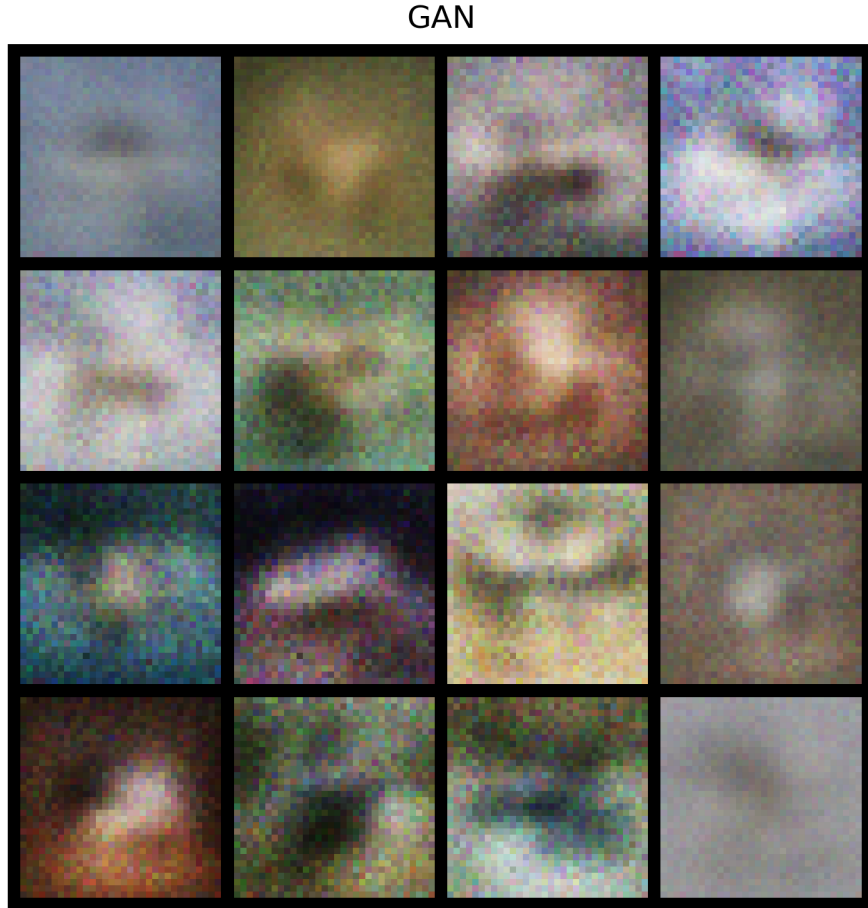


Figure 1: Generated images using the baseline fully-connected GAN.

## 6.2 Deep Convolutional GAN

Replacing fully-connected layers with convolutional operations significantly improved generation quality.

The Generator progressively synthesized images through learned upsampling, while the convolutional Discriminator extracted hierarchical image features.

Compared to the baseline model, DCGAN generated:

- Better object structure.
- More consistent color distribution.
- Sharper edges.
- Improved global coherence.

These improvements demonstrate the effectiveness of convolutional architectures for image synthesis.

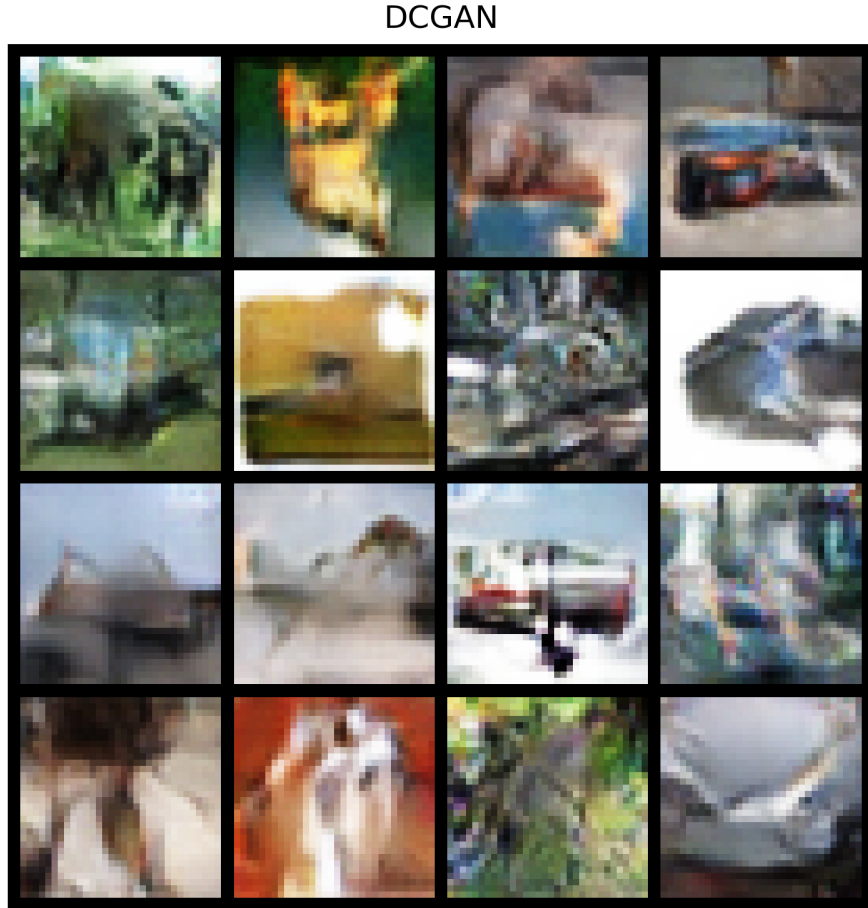


Figure 2: Generated images using DCGAN.

### 6.3 Experiment 1: Spectral Normalization

The first experiment evaluated the impact of Spectral Normalization on the Discriminator.

Rather than modifying the Generator, the pretrained DCGAN Generator was reused while replacing the Discriminator with a Spectral Normalized version.

Training remained noticeably more stable than the baseline DCGAN.

The generated images displayed:

- Improved global structure.
- More consistent object shapes.
- Reduced training instability.
- Smoother optimization behaviour.

Constraining the spectral norm prevented the Discriminator from becoming excessively powerful, allowing the Generator to receive informative gradients throughout training.

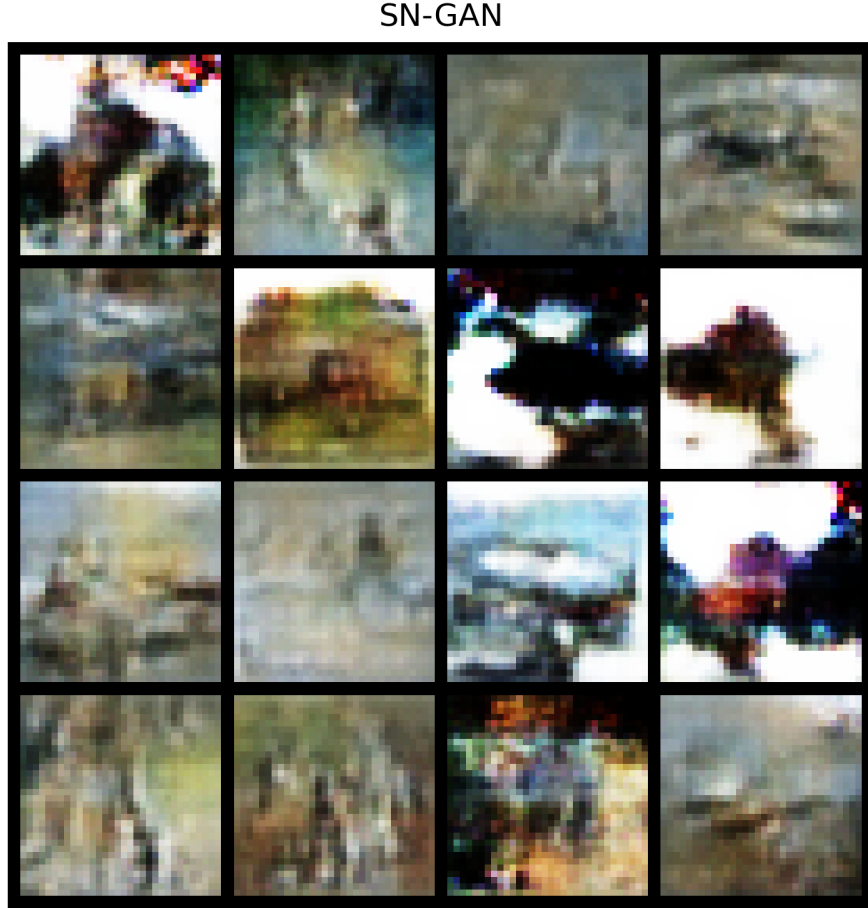


Figure 3: Generated images after fine-tuning with Spectral Normalization.

#### 6.4 Experiment 2: Improved Discriminator

The second experiment focused on increasing the representational capacity of the Discriminator.

Additional convolutional layers enabled richer hierarchical feature extraction, while Dropout regularization reduced overfitting.

Compared to the baseline DCGAN, the modified Discriminator produced:

- Sharper object boundaries.
- Better texture preservation.
- Improved foreground-background separation.
- More detailed object features.

The experiment demonstrates that improving feature extraction within the Discriminator directly benefits Generator learning by providing stronger supervisory signals.

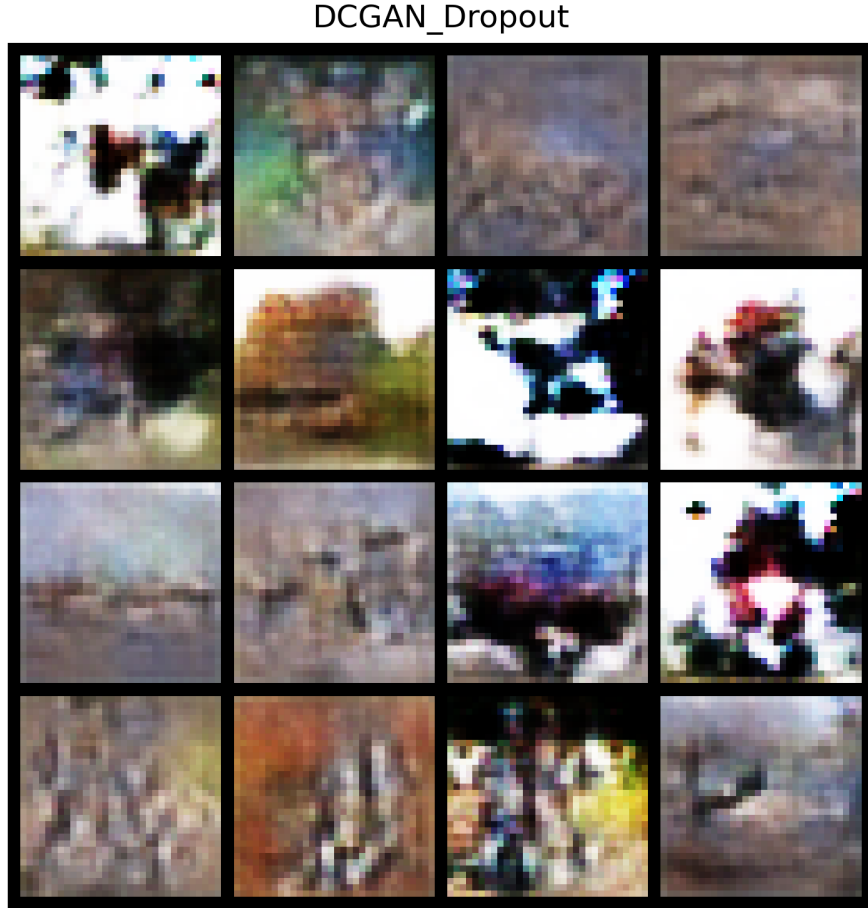


Figure 4: Generated images after fine-tuning using the improved discriminator.

### 6.5 Experiment 3: WGAN-GP

The final experiment replaced the conventional GAN objective with the Wasserstein distance.

Unlike previous models, WGAN-GP employed a Critic rather than a binary classifier and introduced Gradient Penalty to satisfy the Lipschitz constraint.

Among all evaluated models, WGAN-GP produced the highest quality images.

The generated samples demonstrated:

- Strong object coherence.
- Better texture consistency.
- More realistic color distributions.
- Reduced visual artifacts.
- Stable convergence throughout training.

The smoother optimization objective and stronger theoretical foundations of Wasserstein distance significantly improved Generator learning.

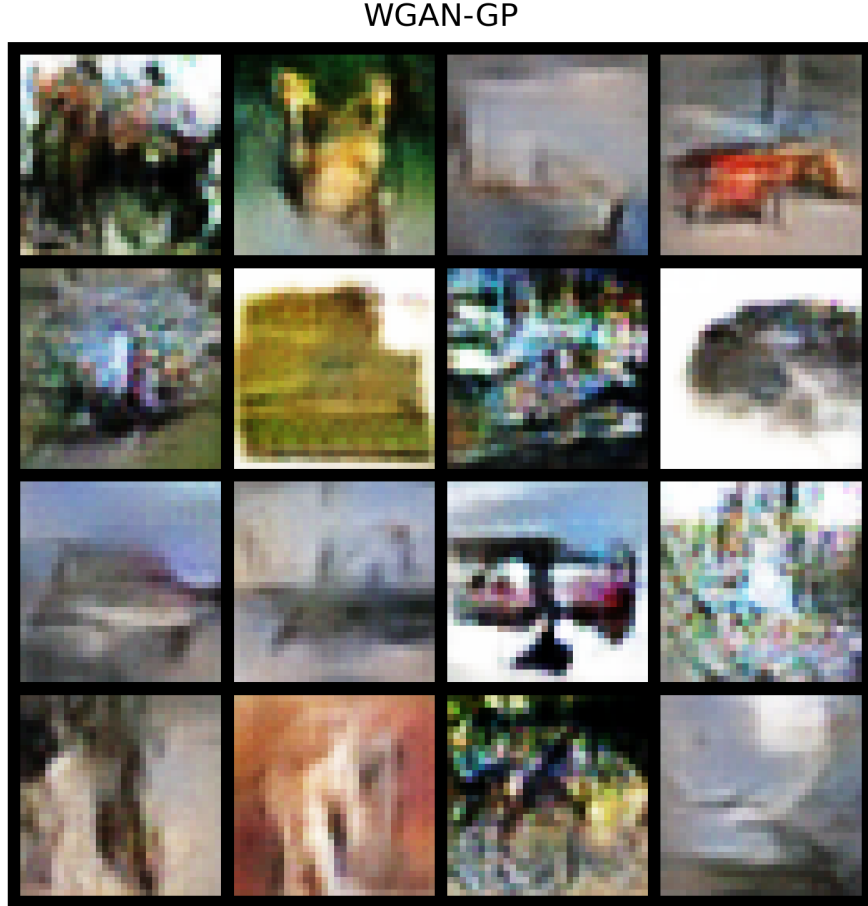


Figure 5: Generated images using WGAN-GP.

## 7 Loss Analysis

Training losses provide additional insight into optimization behaviour.

The baseline GAN exhibited relatively unstable adversarial dynamics due to the imbalance between the Generator and Discriminator.

DCGAN reduced this instability through convolutional feature extraction and Batch Normalization.

Spectral Normalization further stabilized learning by constraining discriminator weights, resulting in smoother convergence.

The improved discriminator maintained competitive Generator and Discriminator losses while producing visually sharper samples.

Unlike conventional GANs, WGAN-GP losses cannot be interpreted as probabilities. Instead, they approximate the Wasserstein distance between the real and generated distributions.

The gradual evolution of both Generator and Critic losses indicated stable optimization without significant mode collapse.

## 8 Comparative Analysis

Table 7 summarizes the qualitative comparison of all evaluated models.

| Model                  | Stability | Image Quality | Texture  | Complexity |
|------------------------|-----------|---------------|----------|------------|
| Baseline GAN           | Low       | Low           | Poor     | Low        |
| DCGAN                  | Good      | Good          | Moderate | Moderate   |
| SN-GAN                 | Very Good | Better        | Good     | Moderate   |
| Improved Discriminator | Very Good | Better        | Better   | Moderate   |
| WGAN-GP                | Excellent | Best          | Best     | High       |

Table 7: Qualitative comparison of implemented GAN models.

Figure 6 presents the generated samples from every model using the same latent vectors.



Figure 6: Visual comparison of generated samples from all implemented models.

From the comparison, several important observations can be made.

1. Fully-connected GANs struggle to model spatial relationships between pixels.

2. Introducing convolutional layers substantially improves visual realism by exploiting local image structure.
3. Spectral Normalization stabilizes adversarial optimization by preventing an overly powerful discriminator.
4. Increasing discriminator capacity enables stronger feature extraction and consequently better supervision for the Generator.
5. WGAN-GP provides the most stable optimization objective and consistently produces the highest quality generated images.

## 9 Conclusion

This assignment investigated multiple Generative Adversarial Network architectures for image generation on the CIFAR-10 dataset.

Beginning with a simple fully-connected GAN established a baseline for comparison. Replacing dense layers with convolutional architectures through DCGAN significantly improved image quality by preserving spatial information during generation and discrimination.

Three independent architectural improvements were subsequently explored.

Spectral Normalization improved training stability by constraining discriminator weights and maintaining meaningful gradients throughout optimization.

The enhanced discriminator architecture demonstrated that stronger feature extraction and regularization provide improved supervision for Generator learning.

Finally, WGAN-GP replaced the original adversarial objective with the Wasserstein distance and Gradient Penalty, resulting in the highest quality generated samples and the most stable convergence among all evaluated models.

Overall, the experiments clearly illustrate that improvements in both architecture and optimization strategy substantially enhance GAN performance.

## 10 Future Work

Several additional extensions could further improve generation quality.

- Conditional GANs for class-specific image generation.
- Self-attention based generators.
- Progressive Growing GANs.
- StyleGAN architectures.
- Higher-resolution image synthesis.
- Quantitative evaluation using Fréchet Inception Distance (FID) and Inception Score (IS).